

Polytechnic University of Puerto Rico  
Biomedical Engineering Department

# Spin It 3D

**Firmware Structure & Safety Features**

Students:

Paola Colon

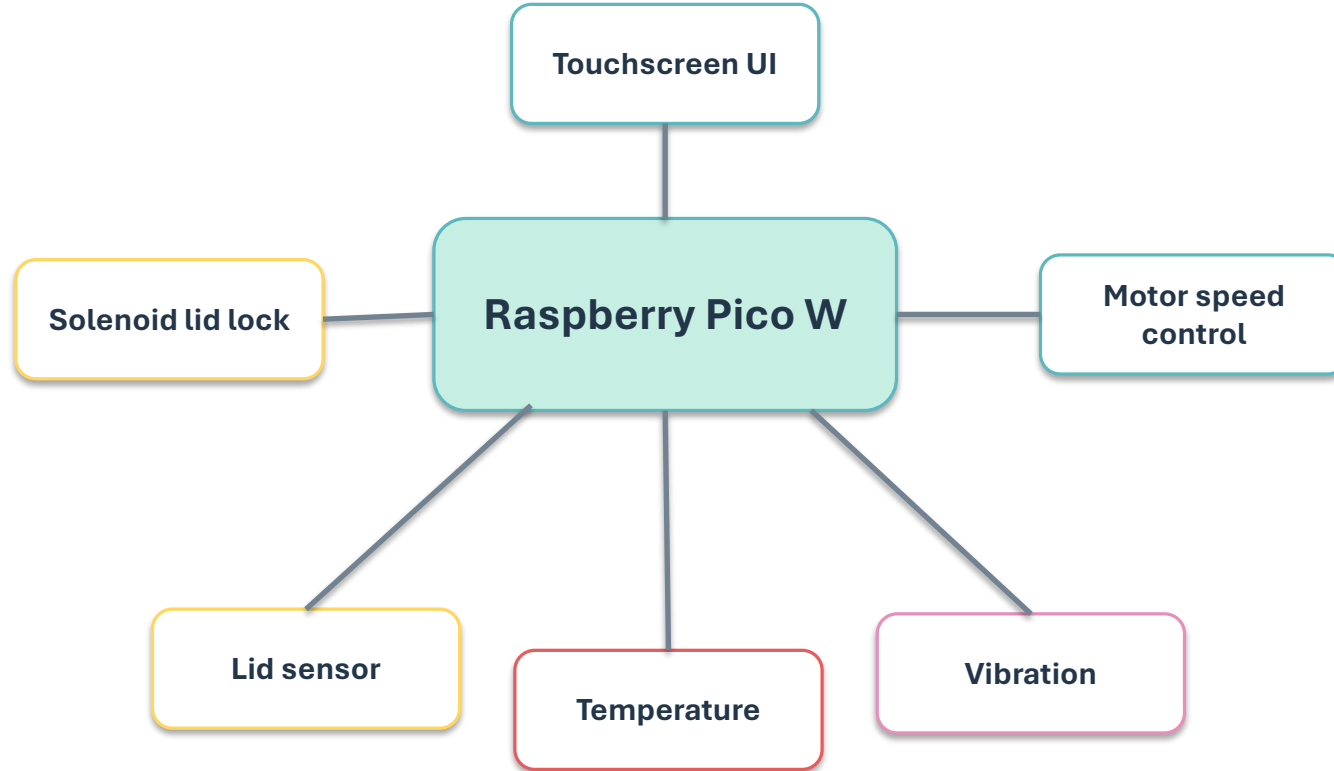
Diana Cortes

Bethzaely Sierra

Jahdiel Perez

Mentors:

Prof. Ricardo Bravo & Dr. Jairo Rondon

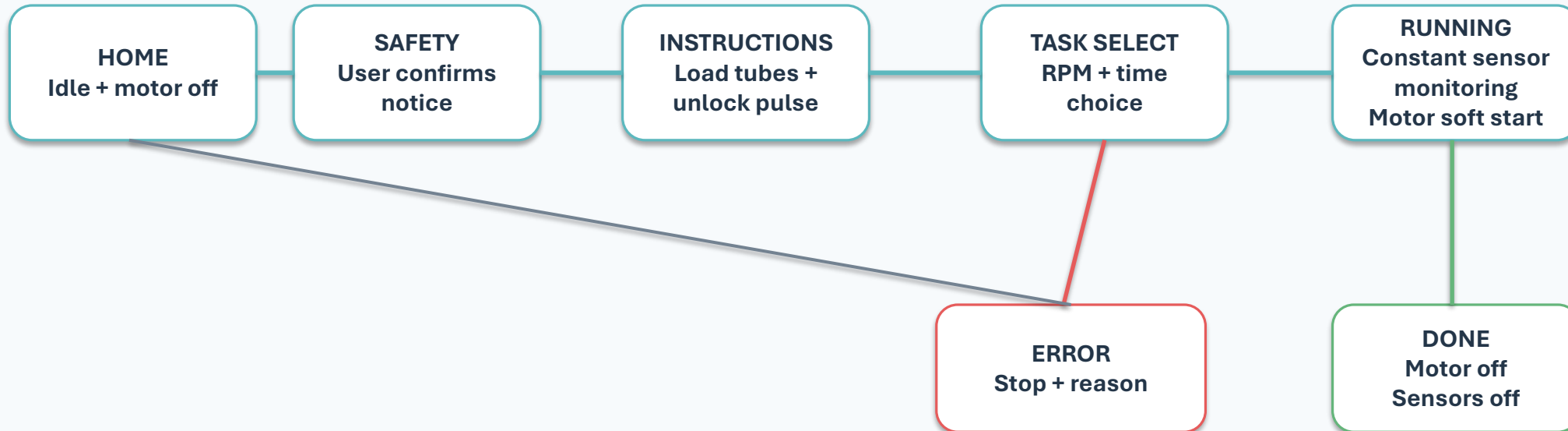


The code is not just a screen menu: it is a supervised controller that only allows motion when the lid, temperature, vibration, timing, and user commands are in safe states.

# Program architecture: simple pages, clear state control

CODE OVERVIEW

The firmware is organized around a screen-state machine. Each page has defined buttons, and each touch moves the system to the next safe step.



## State machine core

```
enum Screen { HOME, SAFETY, INSTRUCTIONS,  
              TASK_SELECT, TASK_RUNNING, TASK_DONE, ERROR };  
Screen currentScreen = SCREEN_HOME;
```

## Why this matters

A state-machine structure makes the workflow predictable: the user cannot accidentally jump straight to spinning without going through safety and task setup.

# What the code connects to

ELECTRICAL I/O

The program treats the Pico as the central safety supervisor handling the UI, the motor speed control, all the sensors, the MOSFET powered solenoid lid latch.

## GP3 | PWM → 0–10V module

motor speed command

## GP13 | Hall-effect lid sensor

detects whether the lid is open or closed

## GP15 | Vibration sensor

counts pulse rate

## GP7 | MOSFET → solenoid

timed solenoid actuation for lid locking and unlocking

## GP26 | Temperature sensor

ADC motor temperature

## GP22/17 | Touch + TFT chip select

shared SPI display system

# Safety is layered into the workflow

## SAFETY LOGIC

Spin It 3D uses checks before starting and continuous monitoring while spinning. Unsafe states lead to a controlled stop and an error message.

### 1. Pre-start gate

Requires RPM AND time selection and verifies the lid is closed, no vibrations are present, and temp is within range before motor power is allowed.

### 2. Live monitoring

The loop continuously checks and refreshes the UI to show lid status, temperature, vibration levels, task time, and RPM.

### 3. Fail-safe response

On an error, the task stops, PWM is set to zero, the stopMotor command is used, and the screen displays what happened.

#### Active safety checks

```
if (!lidClosed()) abortTask("Task halted. Lid is open");
else if (vibrationSurgeDetected)
  abortTask("...vibration...");
else if (temperatureC >= TEMP_C_MAX_SAFE) {
  // must stay high for 5 continuous seconds
}
```

#### Controlled stop behavior

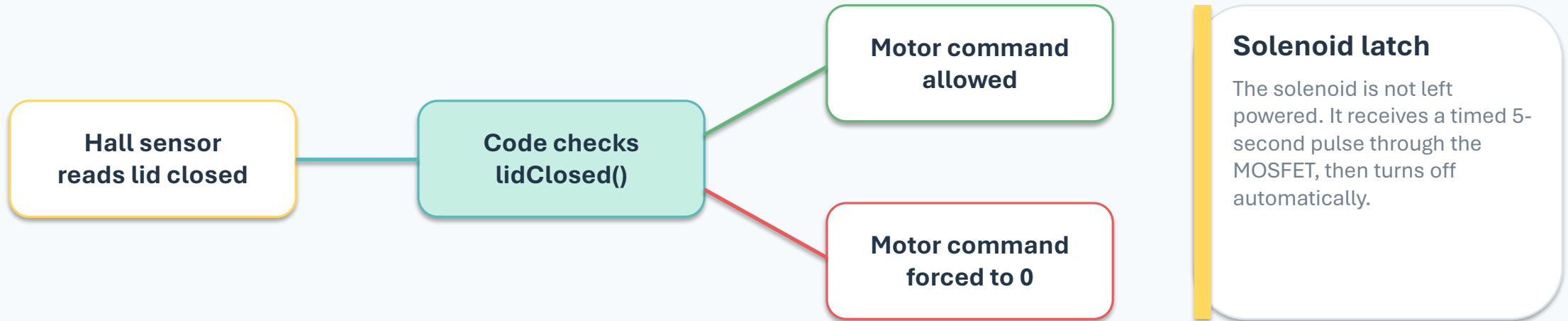
```
void showError(const char *message) {
  taskActive = false;
  stopMotor();
  unlockLid();
  drawErrorMessage(message);
}
```

The display screen printing which error occurred is essential. Clearly defined error messages allow the user to troubleshoot the centrifuge and fix the root of the issues.

# Lid interlock + solenoid latch

SAFETY FEATURE

The lid system is treated as a REQUIRED condition for motion, not just a user-interface warning.



## Motor block

```
if (speedPercent > 0 && !lidClosed())  
    speedPercent = 0; // lid interlock
```

## Latch pulse

```
#define SOLENOID_PULSE_MS 5000  
updateSolenoidPulse(); // auto-off timer
```

# Temperature and vibration safety parameters

SAFETY FEATURE

The code avoids reacting to one noisy sensor spike by averaging temperature and confirming vibration surges before stopping.

## Temperature protection

TMP36-style ADC conversion with 16-sample averaging. A task stops at 65°C only after the limit persists for 5 seconds.

### Temperature parameters

```
#define TEMP_C_MAX_SAFE 65.0
#define TEMP_TRIP_DELAY_MS 5000
#define TEMP_SAMPLE_COUNT 16
return (voltage - 0.5) * 100.0;
```

## Vibration protection

The vibration input is counted by an interrupt, converted to pulse/s, and checked for sustained upward surges.

### Vibration parameters

```
attachInterrupt(..., vibrationPulseISR, RISING);
vibrationRate = pulses * 1000 / elapsed;
#define VIBRATION_SURGE_PPS 100
confirm on the next 1-second reading
```

**Design goal: stop on real sustained risk, not on normal startup noise or faulty readings.**

# Polishing the programming

## CODE HIGHLIGHTS

### Soft-start motor ramp

Starts at 5% and increases by 2% once per second until the selected command is reached.

### Fast display handling

Full-screen image drawing uses TFT\_eSPI, fast SPI transactions, and DMA initialization.

### Efficient screen updates

Only dynamic overlay areas are refreshed each second instead of redrawing the entire UI repeatedly.

### Smooth acceleration

```
#define MOTOR_START_PERCENT 5
#define MOTOR_RAMP_STEP_PERCENT 2
#define MOTOR_RAMP_INTERVAL_MS 1000
```

### UI performance

```
tft.setSwapBytes(true);
tft.initDMA();
tft.pushColors(...);
```

### Clean interface

```
drawRunningOverlay();
progress = elapsed / duration;
restoreArea(button);
```

These choices make the prototype look more like an integrated product: safer startup, responsive controls, and a clear task-running screen.



# What this code demonstrates

SYSTEM VALUE

Spin It 3D's firmware shows an engineering-focused approach: user flow, hardware control, and safety supervision are built into one coordinated controller.

## User-centered workflow

Safety notice, loading instructions, simple task presets, progress display.

## Hardware-aware control

PWM motor command, ADC temperature, interrupt-driven vibration, MOSFET solenoid.

## Safety-first behavior

Lid interlock, timed solenoid pulse, soft-start, over-temp trip, vibration surge stop.

## Clear failure handling

The motor is stopped and the UI gives a specific reason instead of failing silently.

**Bottom line: the program turns the centrifuge into a monitored, guided, and safer electromechanical system.**